

Project Title: **DealWheel – Vehicle Sales Management System**

Milestone: **Project Report**

Group 16

Student 1: Mihir Harishankar Parab

Student 2: Yaswanth Kumar Reddy Gujjula

617-935-6086 (Tel of Mihir)

857-351-8202 (Tel of Yaswanth)

parab.mi@northeastern.edu

gujjula.y@northeastern.edu

Percentage of Effort Contributed by Student 1: 50%

Percentage of Effort Contributed by Student 2: 50%

Signature of Student 1: Mihir Harishankar Parab

Signature of Student 2: Yaswanth Kumar Reddy Gujjula

Submission Date: 8-December 2024

I. Introduction:

The rise of reliability and demand of vehicles in today's world needs a dynamic and flexible data management system. There are various types of vehicles like cars, bikes, buses etc. This vehicles industry focuses on purpose, feature, price. Each vehicle has its own manufacturer which provide features as per customer's needs. A branch plays a main role in selling vehicle, after-sales services. When customer visits the branch, the salesperson suggest vehicles are per customer needs and cooperate with the finance department for payments. SQL database stores structured data like customer information, branch location and many more. On other hand, NOSQL database ensures that our system can handle the volume of the data and semi-and unstructured data.

Theory:

This aims to provide a technological approach for DealWheel – a vehicle sales management system which manages customer requirements, handles inventory and provide services to the customer. This system records each customer's information like name, email etc. For vehicle, it stores it's specification. It also saves salesperson's information like, store name, level, commission etc. The project manages every vehicle's insurance data and after sales maintenance services. The system aims to efficiently manage whole aspects of the vehicle

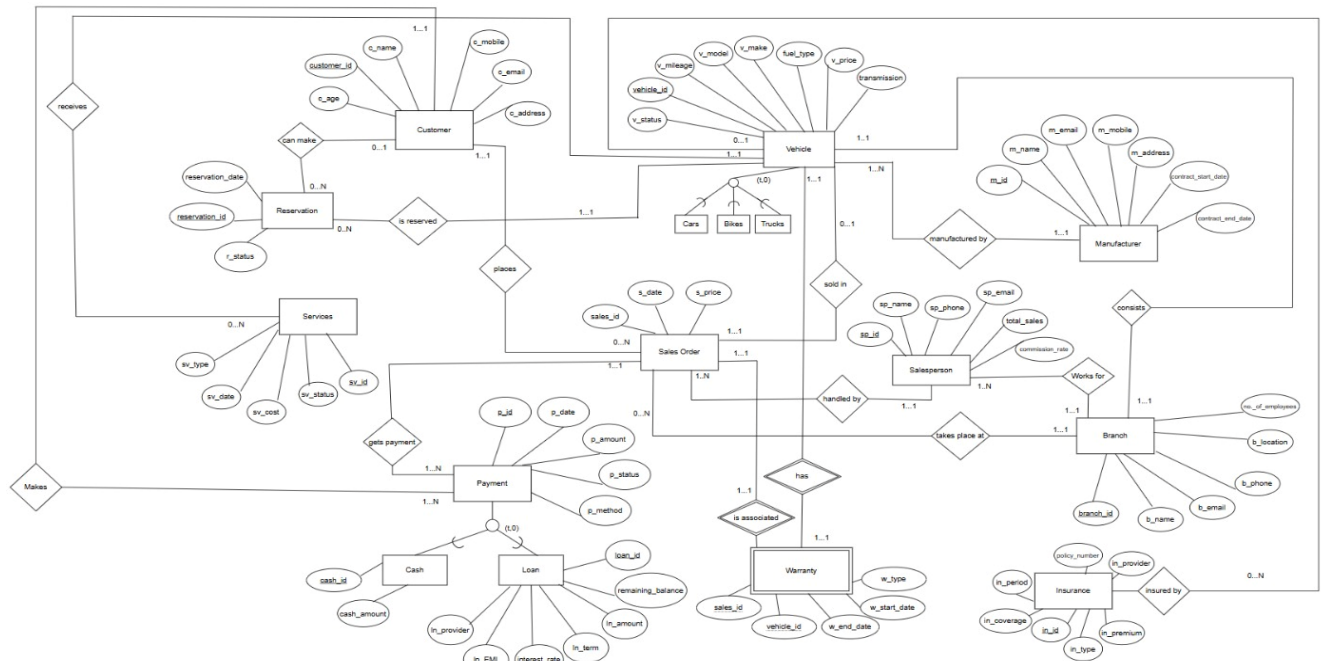
Other requirements:

1. A customer can make multiple reservations, each reservation linked to one customer.
2. A vehicle can undergo multiple services, each service linked to one vehicle.
3. A salesperson can handle multiple sales orders, each order linked to one salesperson.
4. A customer can place multiple sales orders, each order linked to one customer.
5. Each vehicle belongs to one subclass (Car, Bike, or Truck).
6. A manufacturer can produce multiple vehicles, each vehicle linked to one manufacturer.
7. A warranty applies to one sales order, each order linked to one warranty.
8. A vehicle is sold through one sales order, each order linked to one vehicle.

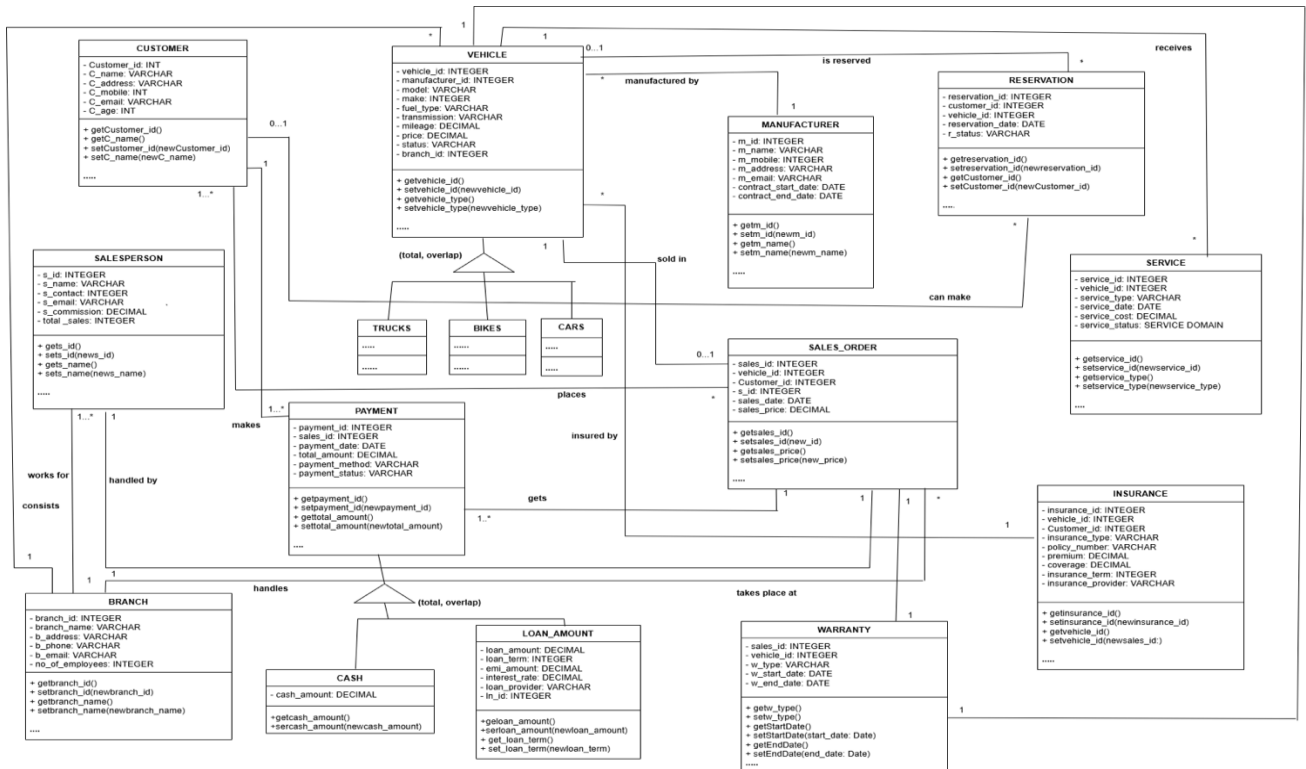
The system provides flexibility to add, edit, delete, and update new data by running different queries on the table. This system is ideal for a real-life application where a company carries all the tasks, right from managing manufacturing to providing services to the purchased vehicle. The database then connected to Python allows you to visualize the data. The insights generated are beneficial for analysis and to implement necessary measures wherever required. Using the metrics, a company can build a relationship with customers who have purchased a vehicle and get feedback from them for improvement.

II. Conceptual Data Modelling

1. Extended Entity Relation Model



2. Unified Modelling Language



III. Mapping Conceptual Model to Relational Model

Note: Primary keys are underlined and foreign keys in *italics*

1. Customer (customer_id, c_name, c_email, c_address, c_mobile, c_age,)

- customer_id – Primary Key

2. Reservation (reservation_id, reservation_date, reservation_status, *customer_id*, *vehicle_id*)

- reservation_id – Primary key
- customer_id – Foreign Key from Customer relation – NULL Allowed
- vehicle_id – Foreign Key from Vehicle relation - NOT NULL

3. Manufacturer (manf_id, manf_name, manf_email, manf_mobile, manf_address, contract_start_date, contract_end_date)

- manf_id – Primary Key

4. Salesperson (sp_id, sp_name, sp_phone, sp_email, total_sales, commission_rate, *branch_id*)

- sp_id – Primary key
- branch_id – Foreign Key from Branch relation – NOT NULL

5. Services (sv_id, sv_status, sv_cost, sv_type, sv_date, *vehicle_id*)

- sv_id – Primary Key
- vehicle_id – Foreign key from Vehicle relation – NOT NULL

6. Vehicle (vehicle_id, v_status, v_make, v_model, v_mileage, v_price, fuel_type, transmission, *manf_id*, *branch_id*)

- vehicle_id - Primary Key
- manf_id – Foreign key from Manufacturer relation – NOT NULL
- branch_id – Foreign Key from Branch relation – NOT NULL

7. Cars (*vehicle_id*)

- vehicle_id – Subclass of Vehicle with vehicle_id as Foreign Key

8. Bikes (*vehicle_id*)

- vehicle_id - Subclass of Vehicle with vehicle_id as Foreign Key

9. Trucks (*vehicle_id*)

- vehicle_id - Subclass of Vehicle with vehicle_id as Foreign Key

10. Branch (branch_id, b_name, b_email, b_phone, b_location, number of employees)

- branch_id – Primary Key

11. Sales Order (sales_id, s_price, s_date, sp_id, customer_id, vehicle_id, branch_id)

- sales_id – Primary Key
- sp_id – Foreign Key from Salesperson relation – NOT NULL
- customer_id – Foreign Key from Customer relation – NOT NULL
- vehicle_id - Foreign Key from Vehicle relation - NOT NULL
- branch_id – Foreign Key from Branch relation – NOT NULL

12. Insurance (insurance_id, ins_type, ins_period, ins_premium, ins_coverage, ins_provider, policy_number, vehicle_id)

- insurance_id – Primary Key
- vehicle_id – Foreign Key from Vehicle relation – NULL Allowed

13. Payment (payment_id, p_amount, p_date, p_status, p_method, sales_id)

- payment_id – Primary Key
- sales_id – Foreign Key from Payment Relation – NOT NULL

14. Cash (cash_id, cash_amount, payment_id)

- cash_id – Primary Key
- payment_id – Subclass of Payment with payment_id as Foreign Key

15. Loan (loan_id, ln_amount, ln_period, remaining balance, ln_emi, ln_term, interest_rate, payment_id)

- loan_id – Primary Key
- payment_id – Subclass of Payment with payment_id as Foreign Key

16. Warranty (vehicle_id, sales_id, w_start_date, w_end_date, w_type)

- vehicle_id, sales_id – Combination of Primary Key
- vehicle_id FK from Vehicle relation and sales_id FK from salesorder relation acting as Primary Key of weak entity of Warranty entity type

IV. Implementation of Relation Model via MySQL and NoSQL

MySQL Implementation: The database was created in MySQL and the following queries were performed

Query1: Give the customer details whose age is greater than 70.

```
SELECT customer_id, c_name, c_email,  
c_address, c_mobile  
FROM Customer  
WHERE c_age >=70;
```

	customer_id	c_name	c_email	c_address	c_mobile
▶	101	Brynn Luna	nibh@protonmail.edu	Ap #626-9720 Velt Av.	(776) 766-2324
	102	Cally Hobbs	quis.tristique@yahoo.ca	771-6924 Felis Ave	(552) 262-8663
	104	Ursula Cox	et.magna@icloud.com	Ap #499-6164 Dui. Road	(705) 576-7726
	110	Veronica Fletcher	sodales.at@protonmail.org	247-5344 Sodales Rd.	(864) 565-1048
	111	Marcia Boyd	ligula.tortor@yahoo.org	383-2430 Metus. Rd.	(523) 641-7645
	112	Zeph White	tempus.risus@icloud.co.uk	P.O. Box 640, 4308 Aliquet Av.	(332) 687-9657
	115	Shoshana Sellers	consequat.purus.maecenas@hotmail.co.uk	997-9396 Ut Avenue	(326) 844-4939
	120	Wylie Blankenship	rhoncus.nulam.velt@hotmail.ca	6587 Phasellus St.	(425) 604-3467
	123	Brenden Battle	eu@yahoo.co.uk	Ap #336-2908 Tortor. Rd.	(272) 866-8572
	126	Elton Byrd	dui.lectus@protonmail.co.uk	3662 Sed Ave	(916) 978-5579
	128	Zane Salas	nunc.ut.erat@yahoo.net	Ap #386-7248 Vehicula Rd.	(345) 605-4821
	129	Hillary Ball	auctor.quis@yahoo.com	P.O. Box 294, 586 Hendreit St.	(927) 220-4314
	137	Dorna Lang	vel.quam.dignissim@yahoo.org	214-626 Et Ave	(275) 833-8646
	138	Nigel Kane	nam.porttitor@hotmail.com	Ap #854-2457 Curabitur St.	(668) 540-2237
	140	Camilla Bryant	dis@hotmail.com	Ap #705-2263 Nam Av.	(867) 852-2893

Query 2: Give the sales metrics of each branch and give branches where total revenue is greater than 150,000.

```
SELECT branch_id,  
COUNT(sales_id) AS TotalSales,  
SUM(s_price) AS TotalRevenue,  
AVG(s_price) AS AverageSalePrice  
FROM Salesorder  
GROUP BY branch_id  
HAVING SUM(s_price) > 150000;
```

	branch_id	TotalSales	TotalRevenue	AverageSalePrice
▶	23	2	248403	124201.5000
	35	3	325784	108594.6667
	34	3	298858	99619.3333
	37	2	302628	151314.0000
	38	2	200767	100383.5000
	27	2	216899	108449.5000
	33	2	191030	95515.0000
	26	2	201063	100531.5000
	25	2	161052	80526.0000
	32	1	169069	169069.0000

Query 3: Retrieve the details of all sales orders along with the corresponding customer name.

```
SELECT so.sales_id, so.s_price, c.c_name  
FROM SalesOrder so  
INNER JOIN Customer c ON  
so.customer_id = c.customer_id  
ORDER BY so.sales_id ASC;
```

	sales_id	s_price	c_name
▶	500	106489	Bianca Dale
	501	113283	Lareina Day
	502	53034	Bianca Dale
	503	71270	Zena Lindsay
	504	193393	Donna Lang
	505	125672	Malik Sampson
	506	51488	Bianca Dale
	507	126328	Cally Hobbs
	508	70179	Bianca Dale
	509	141914	Elton Byrd
	510	109235	Coby Sanford
	511	91620	Zena Lindsay
	512	167945	Brenden Battle
	513	51080	Hillary Ball
	514	85617	Camilla Bryant
	515	106186	Talon Whitfield
	516	159467	Nigel Kane
	517	54866	Malik Sampson

Query 4: Fetch all vehicles and their reservation details, including vehicles without reservations

```
SELECT v.vehicle_id, v.v_status,
r.reservation_id, r.reservation_status
FROM Vehicle v
LEFT OUTER JOIN Reservation r ON
v.vehicle_id = r.vehicle_id;
```

	vehicle_id	v_status	reservation_id	reservation_status
▶	30	sold	67	Unbooked
	30	sold	40	Booked
	31	sold	72	Unbooked
	31	sold	50	Unbooked
	31	sold	49	Booked
	32	unsold	77	Unbooked
	32	unsold	76	Unbooked
	32	unsold	74	Unbooked
	32	unsold	42	Booked
	33	sold	48	Unbooked
	34	sold	NULL	NULL
	35	sold	75	Booked
	35	sold	63	Unbooked
	35	sold	45	Unbooked
	36	sold	NULL	NULL
	37	sold	NULL	NULL
	38	unsold	60	Booked
	38	unsold	43	Booked
	39	unsold	54	Unbooked
	39	unsold	47	Booked
	40	unsold	61	Booked
	41	sold	44	Booked
	42	unsold	NULL	NULL
	43	unsold	59	Unbooked

Query 5: Give the name, email and mobile of all customers who have purchased the most expensive vehicle.

```
SELECT C.c_name, C.c_email, C.c_mobile
FROM Customer C
INNER JOIN SalesOrder S ON
C.Customer_ID = S.Customer_ID
INNER JOIN Vehicle V ON S.vehicle_id =
V.vehicle_id
WHERE V.v_price = (SELECT
MAX(v_price) FROM Vehicle);
```

	c_name	c_email	c_mobile
▶	Cally Hobbs	quis.tristique@yahoo.ca	(552) 262-8663
	Zena Lindsay	velit.in.aliquet@aol.couk	(280) 899-7372

Query 6: Retrieve vehicle and it's make, model whose atleast 1 service is completed

```
SELECT Vehicle.vehicle_id,
Vehicle.v_make, Vehicle.v_model
FROM Vehicle
WHERE EXISTS ( SELECT 1
FROM Services
WHERE Services.vehicle_id =
Vehicle.vehicle_id
AND Services.sv_status = 'Completed' );
```

	vehicle_id	v_make	v_model
▶	55	2017	Safari
	56	2024	Nexon

Query 7: Find vehicles whose total payment received are above the average payment value across all sales

```
SELECT V.vehicle_id, V.v_model,
V.v_price
FROM Vehicle V
WHERE ( SELECT SUM(P.p_amount)
FROM SalesOrder S
INNER JOIN Payment P ON S.sales_id =
P.sales_id
```

	vehicle_id	v_model	v_price
▶	30	GLC	245323
	33	GLC	125465
	34	CBR	83123
	45	Amaze	170304
	47	Nexon	107969
	48	AMG	125544
	50	S60	196145
	53	GLC	188118
	54	X5	148748
	57	V60	150108
	59	V60	147487

```
INNER JOIN Payment P ON S.sales_id =
P.sales_id
WHERE S.vehicle_id = V.vehicle_id ) >
( SELECT AVG(P2.p_amount)
FROM Payment P2 );
```

Query 8: Find vehicles that have been reserved on any reservation date before '07/01/2024'

```
SELECT V.vehicle_id, V.v_model,
V.v_make
FROM Vehicle V
WHERE V.vehicle_id = ANY ( SELECT
R.vehicle_id
FROM Reservation R
WHERE R.reservation_date <
'07/01/2024' );
```

	vehicle_id	v_model	v_make
▶	31	GLC	2017
	32	X5	2015
	35	X3	2020
	38	AMG	2022
	39	S60	2016
	40	S60	2020
	43	CBR	2018
	44	Safari	2022
	47	Nexon	2020
	50	S60	2016
	52	X3	2022
	54	X5	2018
	55	Safari	2017

Query 9: List all sales orders with the number of vehicles sold in each order as an additional column

```
SELECT S.sales_id, S.s_price, (SELECT
COUNT(V.vehicle_id)
FROM Vehicle V
WHERE V.vehicle_id = S.vehicle_id) AS
NumberOfVehicles
FROM SalesOrder S;
```

	sales_id	s_price	NumberOfVehicles
▶	500	106489	1
	501	113283	1
	502	53034	1
	503	71270	1
	504	193393	1
	505	125672	1
	506	51488	1
	507	126328	1
	508	70179	1
	509	141914	1
	510	109235	1
	511	91620	1
	512	167945	1
	513	51080	1
	514	85617	1
	515	106186	1
	516	159467	1
	517	54866	1

Query 10: Combine customers who have either made a reservation or purchased a vehicle.

```
SELECT C.customer_id, C.c_name,
'Reservation' AS InteractionType
FROM Customer C
INNER JOIN Reservation R ON
C.customer_id = R.customer_id
UNION
SELECT C.customer_id, C.c_name,
'SalesOrder' AS InteractionType
FROM Customer C
INNER JOIN SalesOrder S ON
C.customer_id = S.customer_id;
```

	customer_id	c_name	InteractionType
▶	103	Demetria Tanner	Reservation
	104	Ursula Cox	Reservation
	107	Ivy Hoffman	Reservation
	112	Zeph White	Reservation
	113	Bianca Dale	Reservation
	115	Shoshana Sellers	Reservation
	118	Malik Sampson	Reservation
	120	Wylie Blankenship	Reservation
	121	Kelsie Mdean	Reservation
	122	Keelie Kinney	Reservation
	123	Brenden Battle	Reservation
	125	David Harrison	Reservation
	126	Elton Byrd	Reservation
	128	Zane Salas	Reservation
	129	Hillary Ball	Reservation
	131	Maxine England	Reservation

NoSQL Implementation:

A new database connection in the name of "DB" is created in MongoDB and all the necessary collections have been created for the NoSQL implementation.

Query 1: Find Insurance Details for Vehicles with "Collision Cover"

db.insurance.find({ ins_type: "collision cover" })

```
> use DB
< switched to db DB
> db.insurance.find({ ins_type: "collision cover" })
< {
  _id: ObjectId('6750e8366279c701b930aca7'),
  insurance_id: 62,
  ins_type: 'collision cover',
  ins_period: 9,
  ins_premium: 5127,
  ins_coverage: 80,
  ins_provider: 'StateAuto',
  policy_number: 'Q8K 4B5',
  vehicle_id: 145
}
{
  _id: ObjectId('6750e8366279c701b930acab'),
  insurance_id: 66,
  ins_type: 'collision cover',
  ins_period: 2,
  ins_premium: 3555,
  ins_coverage: 33,
  ins_provider: 'StateAuto',
  policy_number: 'Z3J 1XS',
  vehicle_id: 112
}
```

Query 2: Find all vehicles with a fuel type of 'Hybrid' or 'Electric' that have a price greater than 100,000, and sort the results in descending order of price.

```
db.vehicle.find(
  {
    fuel_type: { $in: ["Hybrid", "Electric"] },
    v_price: { $gt: 100000 }
  },
  {
    _id: 0,
    v_make: 1,
    v_model: 1,
    v_price: 1,
    fuel_type: 1
  }
).sort({ v_price: -1 });
```

```
< {
  v_make: 2017,
  v_model: 'S60',
  v_price: 218904,
  fuel_type: 'Electric'
}
{
  v_make: 2024,
  v_model: 'Nexon',
  v_price: 214219,
  fuel_type: 'Electric'
}
{
  v_make: 2019,
  v_model: 'Accord',
  v_price: 188768,
  fuel_type: 'Electric'
}
```

Query 3: Retrieve the details of vehicles sold by the branch with branch_id = 29, including their vehicle_id and all associated vehicle information, using MongoDB's aggregation pipeline

```
db.salesorder.aggregate([
  {
    $match: { branch_id: 29 }
  },
  {
    $lookup: {
      from: "vehicle", // Replace with the
      actual collection name storing vehicle
      details
      localField: "vehicle_id",
      foreignField: "vehicle_id",
      as: "vehicle_details"
    }
  },
  {
    $project: {
      _id: 0,
      vehicle_id: 1,

```

```

    vehicle_details: 1
  }
}
])
{
  vehicle_id: 31,
  vehicle_details: [
    {
      _id: ObjectId('6750e7296279c701b930ac75'),
      vehicle_id: 31,
      v_status: 'unsold',
      v_make: 2019,
      v_model: 'Accord',
      v_mileage: 26,
      v_price: 188768,
      fuel_type: 'Electric',
      transmission: 'DCT',
      manf_id: 11,
      branch_id: 29
    }
  ]
}
]

```

Query 4: Find all vehicles that have completed services.

```

db.services.aggregate([
{
  $match: { sv_status: "Completed" }
},
{
  $lookup: {
    from: "vehicle",
    localField: "vehicle_id",
    foreignField: "vehicle_id",
    as: "vehicle_details"
  }
},
{
  $project: {
    _id: 0,
    sv_id: 1,
    sv_status: 1,
    vehicle_details: 1
  }
}
]

```

```

])
< {
  sv_id: 54,
  sv_status: 'Completed',
  vehicle_details: [
    {
      _id: ObjectId('6750e7296279c701b930ac7c'),
      vehicle_id: 38,
      v_status: 'unsold',
      v_make: 2024,
      v_model: 'Nexon',
      v_mileage: 11,
      v_price: 214219,
      fuel_type: 'Electric',
      transmission: 'CVT',
      manf_id: 19,
      branch_id: 21
    }
  ]
}

```

Query 5: Find reservations with Booked status for customers older than 40

```

db.reservation.aggregate([ { $lookup:
{ from: "customer", localField:
"customer_id", foreignField: "customer_id",
as: "customer_info" } },
{ $unwind: "$customer_info" },
{ $match: { reservation_status: "Booked",
"customer_info.c_age": { $gt: 40 } } } ])

```

```

{
  _id: ObjectId('6750e9646279c701b930acce'),
  reservation_id: 49,
  reservation_date: '10/21/2023',
  reservation_status: 'Booked',
  customer_id: 110,
  vehicle_id: 47,
  customer_info: {
    _id: ObjectId('6750e3506279c701b930ac45'),
    customer_id: 110,
    c_name: 'Cherokee Hester',
    c_email: 'libero.lacus@yahoo.edu',
    c_address: 'P.O. Box 169, 9743 Id Rd.',
    c_mobile: '(361) 610-4650',
    c_age: 56
  }
}
]

```

V. Database access via Python

Connecting to database:

```
[ ] # Database connection details
db_name = "vehicle_sales_management_system"
db_host = "localhost"
db_username = "root"
db_password = "mihir30gp"

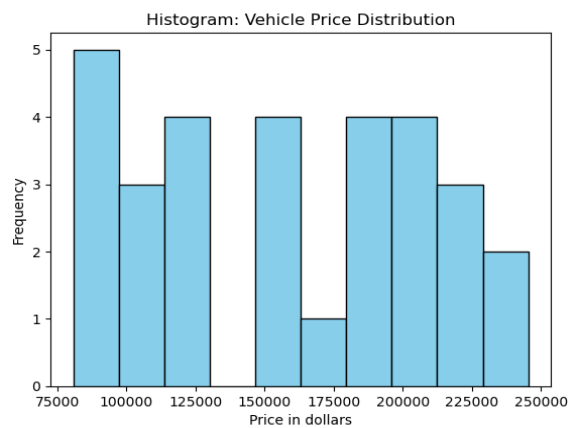
[ ] !pip install pymysql
Requirement already satisfied: pymysql in d:\anaconda\lib\site-packages (1.1.1)

[ ] import pymysql

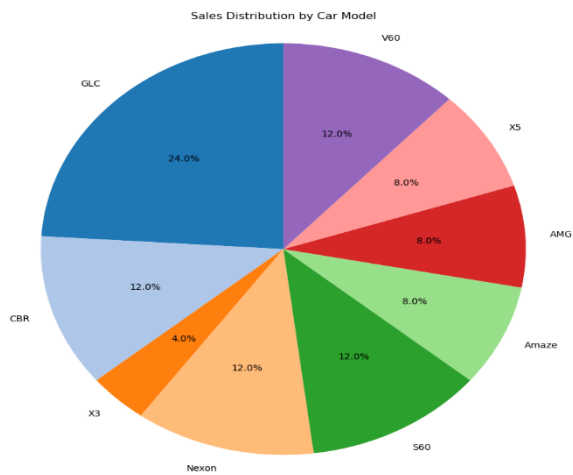
[ ] conn = pymysql.connect(
    host=db_host,
    port=3306,
    user=db_username,
    passwd=db_password,
    db=db_name
)
print("Connection successful!")

[ ] cursor = conn.cursor()
cursor.execute("SHOW TABLES;")
tables = cursor.fetchall() # Fetch all table names
cursor.close()
```

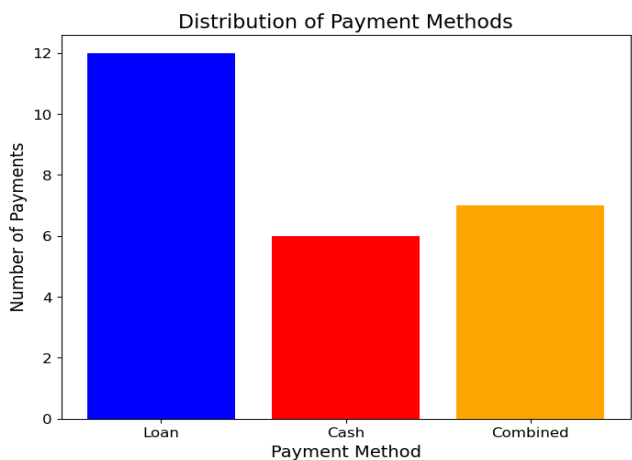
Query 1: A histogram showing distribution of vehicle prices



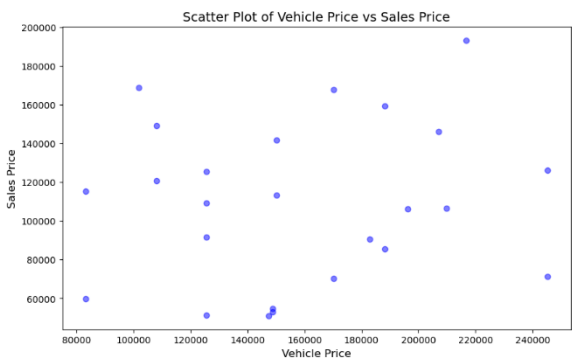
Query 2: A pie chart representing sales distribution by car model



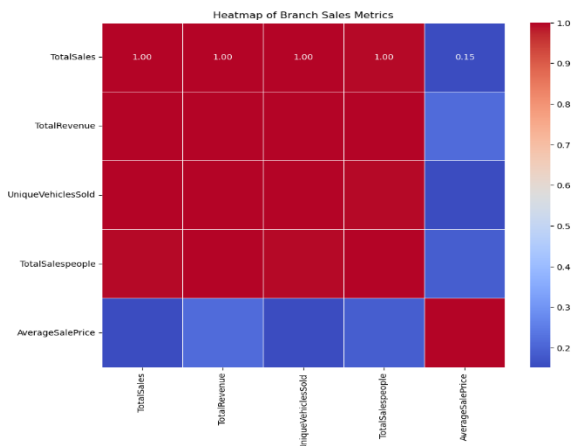
Query 3: A bar chart showing the payment type opted by customers



Query 4: A scatter plot visualising vehicle sales price and its manufacturing year.



Query 5: A heatmap to visualize the relationship between SalesOrder, Payment and Customer.



VI. Summary and Recommendations

DealWheel Vehicle Sales Management System is a database solution that handles vehicle sales, inventories, customer issues, and after-sales service. It will make use of MySQL for structured data and MongoDB for semi-structured and unstructured data, providing customer management, vehicle management, reservations, sales orders, and payment handling. It categorizes the vehicle into cars, bikes, and trucks, supports multiple modes of payment, and integrates the data with the help of Python. Its analytics need to be beefed up, integrated with more advanced tools to study customer behavior, predict vehicle maintenance, and implement the MySQL and MongoDB solutions on the cloud. Future enhancements may also include real-time tracking inventories, predictive sales analytics, and automation of warranty management. Third-party integrations, such as with insurance providers, vehicle registries, and automation of mundane tasks, including insurance renewals and service reminders, will make this system strong, scalable, and efficient in the competitive market.